

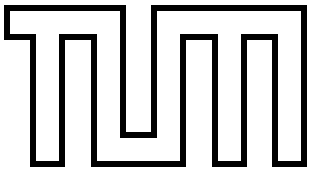
SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY

DER TECHNISCHEN UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Optimisation and Evaluation of Communication and Load Balancing Strategies in MD-Flexible

Ashutosh Solanki



SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY

DER TECHNISCHEN UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Optimisation and Evaluation of Communication and Load Balancing Strategies in MD-Flexible

Author:	Ashutosh Solanki
Supervisor:	Univ.-Prof. Dr. Hans-Joachim Bungartz
Advisor:	Samuel James Newcome, M.Sc.
Date:	25.08.2025

I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, 25.08.2025 Ashutosh Solanki

Acknowledgements

First and foremost, I would like to thank my advisor, Sam, whose steady guidance and encouragement were invaluable throughout the span of this thesis. His readiness to discuss my many ideas in our meetings and his patience in answering my many questions made a tremendous difference.

I am also indebted to the Chair of Scientific Computing and to Prof. Dr. Hans-Joachim Bungartz for offering me the opportunity to pursue this work.

Furthermore, I gratefully acknowledge the Leibniz Supercomputing Centre for supporting this project by granting access to computing time on its Linux-Cluster.

On a personal note, I am deeply thankful to my family, whose unwavering support and encouragement have accompanied me on every step of my academic journey. I am equally grateful to my friends, who patiently endured months of thesis-related conversations. A special mention goes to my friend Johannes and my brother Anirudh for lending their time to proofread and polish my writing.

Abstract

Molecular Dynamics (MD) simulations are widely used to study particle interactions in biology and materials science. Large-scale simulations face performance challenges due to particle migration across domain boundaries, causing load imbalance and communication overhead. This thesis improves `MD-Flexible`, a simulation demo based on `AutoPas`, by introducing a non-blocking MPI communication scheme and parameter-tuned load balancing. These changes enhance efficiency, enabling faster simulations of large particle systems.

Contents

Acknowledgements vii

Abstract ix

I. Introduction and Background 1

1. Introduction 2

2. Background 3
2.1. Molecular Dynamics . . . 3
2.2. Autopas . . . 3
2.3. MD-Flexible . . . 4
2.4. Message Parsing Interface (MPI) . . . 6
2.5. Tuning and Analysis Utilities (TAU) . . . 7
2.6. Load Balancing . . . 7
2.6.1. Domain Decomposition . . . 7
2.6.2. Regular Grid Decomposition . . . 8
2.6.3. ALL-library . . . 9
2.6.4. Inverse Pressure Load Balancer . . . 9
2.6.5. Workload Constraints . . . 10

II. Related Work 11

3. Grid-Based Decompositions 12
3.1. Staggered Grids . . . 12
3.2. Recursive Coordinate Bisection (RCB) . . . 12
3.3. Zoltan Load Balancer . . . 13
3.4. Empirical Successes . . . 14
3.5. Limitations and Challenges . . . 14

III. Implementation and Evaluation 16

4. Experimental Setup 17

5. MPI-Based Optimization 18
5.1. Motivation and Rationale . . . 18

5.2. Implementation Details . . . 18
5.2.1. Original Implementation . . . 18
5.2.2. New Implementation . . . 19
5.3. Experimental Results . . . 20
5.4. Qualitative Evaluation . . . 21
5.5. Interpretation of Results . . . 22

6. Parameter-Tuned Load Balancing **23**
6.1. Motivation and Rationale . . . 23
6.2. Implementation Details . . . 23
6.2.1. Original Implementation . . . 23
6.2.2. New Implementation . . . 23
6.3. Experimental Results . . . 24
6.4. Qualitative Evaluation . . . 25
6.5. Interpretation of Results . . . 27

IV. Conclusion **29**

7. Key Findings and Future Directions **30**
7.1. Summary and main conclusions . . . 30
7.2. Future work . . . 30

V. Appendix **32**

8. Additional Material **33**
8.1. Baseline MD-Flexible Results . . . 33
8.2. Code and YAML Listings . . . 33
8.2.1. YAML Test Configuration . . . 33
8.2.2. Simulation Algorithms . . . 34

Bibliography **40**

Part I.

Introduction and Background

1. Introduction

With the rapid advancements in computational power and parallel architectures, Molecular Dynamics (MD) simulations have become an essential tool in modern scientific research. Their strength lies in capturing the motion and interactions of particles at a microscopic scale, offering insights that would otherwise be inaccessible or expensive through physical experiments. Applications of MD span a wide range of disciplines: in biology, they are employed to investigate protein folding [1] and support drug discovery pipelines, while in materials science, they are used extensively in semiconductor research [2].

A central challenge of MD simulations lies in the massive number of particles required to obtain meaningful results. To scale up to thousands of particles, domain decomposition is commonly applied: the simulation domain is partitioned among MPI ranks, each responsible for a subset of the particles. Since particles naturally move under the influence of forces, they frequently cross sub-domain boundaries. This continuous migration introduces load imbalance, as particle distributions across ranks can become uneven over time, significantly degrading performance if left unaddressed.

`MD-Flexible` is a molecular dynamics simulation demo built on top of the `AutoPas` library. It currently employs regular grid decomposition that, while robust and easy to implement, restricts flexibility in balancing loads and can result in worse performance. Two central research questions, therefore, arise:

1. How can MPI communication for particle exchange across sub-domains be improved?
2. How can load balancing strategies be designed to adapt effectively to dynamic particle distributions?

This thesis addresses these questions in two steps. First, the traditional probe-based particle exchange mechanism is replaced by a non-blocking, tag-structured communication scheme. Second, new options are introduced to guide load balancing, allowing the user to prioritise different metrics when redistributing particles.

In summary, this thesis investigates how `MD-Flexible` can be made more efficient by refining both its communication and load balancing mechanisms, thereby enabling more scalable and robust MD simulations.

2. Background

Molecular dynamics (MD) simulations, such as those performed using `MD-Flexible`, are computationally intensive and are often executed on massive parallel systems. These simulations offer deep insights into how particles behave under different conditions and forces, but their scalability and efficiency are heavily influenced by how well the computational workload is distributed across available hardware resources.

Load balancing and Message Passing Interface are critical factors in determining the overall performance of such parallel simulations. While poor load distribution leaves compute units idle and reduces resource utilisation, inefficient MPI communication can introduce significant synchronisation overhead and latency, further increasing the time-to-solution.

This chapter aims to provide the necessary background to understand the context behind the work done in this thesis.

2.1. Molecular Dynamics

Molecular dynamics (MD) is a computational technique used to simulate the physical behaviour of systems at the atomic and molecular scale. It allows us to study and visualise the molecular properties in terms of their structure, dynamics, and microscopic interactions. These interactions are governed by classical mechanics and intermolecular forces.

MD simulations have become indispensable in many scientific domains, including physics, chemistry, materials science, and biology, as showcased in [3]. They enable researchers to model environments that are either impossible or impractical to recreate in laboratory conditions.

While MD offers powerful insights, it also imposes significant computational demands. The number of interactions that must be computed at each time step scales with the number of particles in the system, and achieving meaningful results requires stepping through thousands of such time steps. As a result, optimising performance and parallel efficiency is critical, especially for large-scale simulations.

2.2. Autopas

`AutoPas` is an open-source C++ library designed to deliver optimal node-level performance for short-range particle simulations through automatic runtime tuning of algorithmic configurations [4]. It has been developed at the Technical University of Munich by the Chair for Scientific Computing in Computer Science. Functionally, `AutoPas` acts as a black-box data container that abstracts the complexity of particle updates, node-level optimisations, and shared memory parallelisation. Users primarily interact with `AutoPas` by initialising particle properties, accessing particles, and defining short-range force interactions. Beyond

the container abstraction, AutoPas also provides a particle class and a functor interface tailored for the Lennard-Jones potential, which is commonly used in molecular dynamics [5].

To minimise the number of unnecessary distance checks when computing pairwise interactions, AutoPas supports multiple optimisation strategies such as using Linked Cells, Verlet Lists or a combination of both. The selection among these methods is handled dynamically during the tuning phases, in which different strategies are benchmarked over several iterations. The configuration with the best performance is then retained for the following simulation steps, until the next tuning phase is triggered.

In simulations employing MPI parallelisation, users can optionally enable a feature that allows AutoPas containers on different nodes to share their tuning results. These results contribute to a shared knowledge base, enabling other containers to make informed optimisation decisions based on collective performance data [5].

AutoPas is used in massively parallel simulations as demonstrated by its integration into MD-Flexible (see Section 2.3).

MD-Flexible interacts with AutoPas using a defined set of interface functions. These include addParticle(), addOrUpdateHaloParticle(), updateContainer(), resizeBox(), getNumberOfParticles(), iteratePairwise(), and getCutoff(). These functions allow the simulation to manage particle insertion and updates, handle domain resizing, and perform pairwise force evaluations [5].

Comprehensive documentation for all interface functions is available in the source code and on the official GitHub repository[1].

2.3. MD-Flexible

MD-Flexible is a demonstration application developed at the Technical University of Munich by the Chair for Scientific Computing in Computer Science. It is designed to showcase the integration of AutoPas into a full-fledged Molecular Dynamics (MD) simulation environment. Developed alongside the AutoPas library, MD-Flexible is used not only to validate AutoPas' performance potential, but also to provide a reference implementation that other users can adapt in their own research.

The application supports several predefined simulation scenarios: for example, particle clusters arranged as spheres or blocks with particles positioned uniformly, on a grid. Users may also initialise simulations from previously recorded states, which are saved in YAML format. Configuration is highly flexible and can be provided either using the command-line arguments or using YAML files. This allows for the definition of particles, AutoPas tuning parameters, and simulation properties, which are tailored to each scenario.

MD-Flexible is capable of simulating a variety of physical scenarios. Three representative cases are discussed below, each illustrating different challenges in terms of load balancing and MPI communication patterns.

- **Falling Drop**, where a dense spherical droplet falls into a bulk liquid created from a grid of tightly packed particles; an "infinite mass" grid layer acts as a fixed floor and gravity is simulated onto the falling droplet and bulk liquid particles. As the droplet impacts the liquid, strong local density variations arise, creating uneven force

[1] https://github.com/AutoPas/AutoPas

computation across sub-domains. This scenario, therefore, stresses the need for efficient halo exchanges near the dense droplet region (see Figure 2.1).

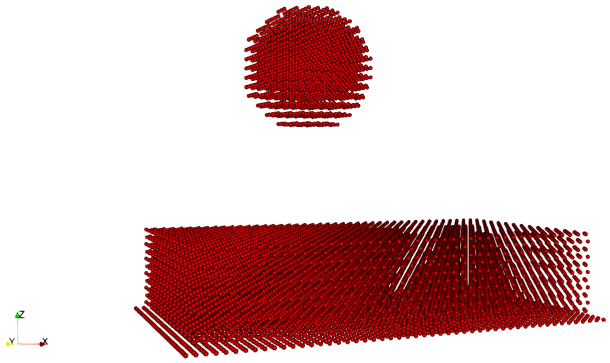


Figure 2.1.: Falling Drop simulation snapshot [5].

- **Spinodal Decomposition**, in which the system reaches equilibrium over one million iterations and then undergoes phase separation under temperature changes. Here, the long runtime emphasises the importance of balanced work distribution over extended periods. Since the phase separation gradually changes particle distributions, load imbalance can accumulate if the balancer reacts too slowly. MPI overhead also becomes critical due to frequent halo exchanges across many iterations (see Figure 2.2).

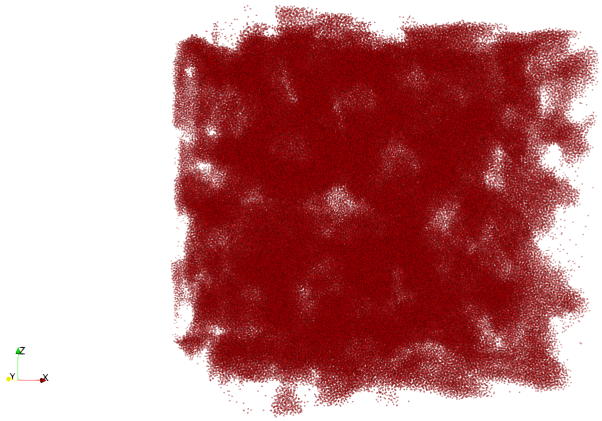


Figure 2.2.: Spinodal simulation snapshot [5].

- **Exploding Liquid**, which simulates a rapidly expanding liquid starting from a densely packed particle block. The fast-changing spatial distribution makes this scenario particularly challenging for both MPI communication and load balancing. Domains that

begin with high density quickly become sparse, while others are overloaded, requiring frequent rebalancing. Communication overhead also increases sharply, as particles migrate across many neighbouring domains within a short time (see Figure 2.3).

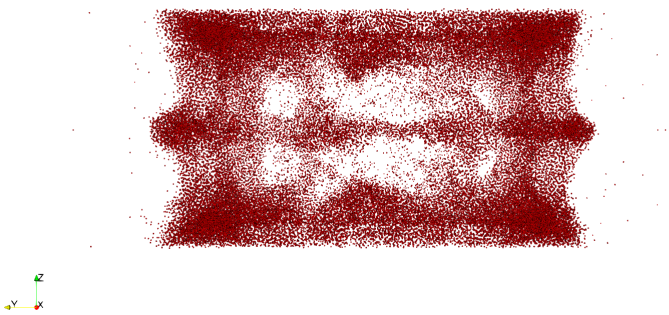


Figure 2.3.: Exploding Liquid simulation snapshot [5].

MD-Flexible can be executed in both single-process and multi-process (MPI) modes. In MPI mode, the global domain is decomposed into a regular grid pattern, generated based on the number of ranks provided. Each MPI process handles its assigned sub-domain. Users can restrict decomposition to specific axes (for instance, excluding the z-axis when gravity dominates in that direction), enabling implicit load balancing aligned with physical forces, which has been shown to provide better performance results.

2.4. Message Parsing Interface (MPI)

The Message Passing Interface (MPI) is a standardised and portable message-passing system designed to facilitate communication between processes in parallel computing environments. Since its initial release in 1994, MPI has become the de facto standard for distributed memory parallel programming in high-performance computing (HPC) [6].

In essence, MPI enables different processes (often distributed across multiple computing nodes) to exchange data explicitly through message passing. These processes are uniquely identified by their rank within a communicator. A communicator is a key abstraction in MPI used to group processes and manage collective operations.

MPI supports a rich set of features essential for HPC:

- Point-to-point communication (send/receive)
- Collective operations (broadcast, gather, scatter, all-to-all)
- Explicit synchronisation (barriers)

One of MPI's key strengths is its flexibility and low-level control, allowing developers to fine-tune performance-critical applications like Molecular Dynamics (MD) simulations.

However, this flexibility comes at a cost: developers must carefully balance communication overhead with redundant computations.

In `MD-Flexible`, MPI is used to support parallel simulations, where the simulation domain is divided into sub-domains distributed across multiple MPI ranks.

2.5. Tuning and Analysis Utilities (TAU)

Tuning and Analysis Utilities (TAU) is a portable and scalable profiling and tracing toolkit designed for the performance analysis of parallel programs written in languages such as Fortran, C, C++, Java, and Python [7]. TAU enables detailed investigation of application behaviour on high-performance computing systems, and has been developed collaboratively by the University of Oregon, Los Alamos National Laboratory, and Forschungszentrum Jülich.

Profiles summarise performance metrics (e.g. execution time, hardware counter values) across processes and threads, while traces capture detailed event sequences. Profiles can be visualised with TAU's ParaProf tool or exported to third-party visualisers. This enables quick identification of bottlenecks, communication imbalances, and scalability issues.

2.6. Load Balancing

In `MD-Flexible`, load balancing plays a critical role in optimising performance. The objective of load balancing is to ensure that all available computational resources are utilised efficiently by distributing the simulation workload evenly.

2.6.1. Domain Decomposition

Domain decomposition is a technique used to distribute work across multiple processes or ranks. The domain of the simulation is first subdivided into smaller sub-domains and then each is assigned to a rank, enabling these sub-domains to be processed concurrently. In `MD-Flexible`, domain decomposition is tightly coupled with MPI-based parallelisation and is primarily there to balance the load and manage the inter-process communication.

Once the domain is subdivided, each process only has knowledge of and direct access to particles within its assigned region. However, correct physical simulations require knowledge of all the neighbouring particles, and in the case of `MD-Flexible`, those within the cut-off radius. This necessitates communication between processes to exchange particle information near sub-domain boundaries.

`MD-Flexible` implements this via *halo particles*. These are particles that physically belong to neighbouring sub-domains, but are locally replicated to enable correct force calculations across boundaries. The extent of the halo region is defined by the cut-off radius and an optional skin width (see Figure 2.4). At every time step, processes exchange halo particles to maintain consistency and ensure accurate interactions near sub-domain edges.

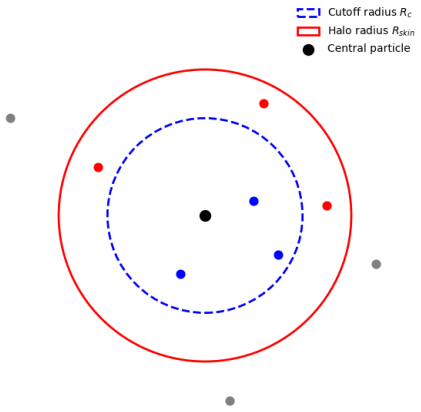


Figure 2.4.: Illustration of the cut-off radius and skin width. The reference particle is shown in black; directly accessible particles are in blue; halo particles, which must be imported, are in red; and inaccessible particles are in grey. Adapted from [5].

While straightforward, this approach has several drawbacks:

- **Memory Overhead:** Halo particles increase the number of particles each rank must store and handle, which increases memory usage and computational workload.
- **Redundant Computations:** Newton's third law ($F_{ij} = -F_{ji}$) is not utilised, leading to duplicated force calculations across ranks.
- **Increased Communication:** These Halo particles must have values equal to their real counterparts and hence need to be kept updated each time step, which adds additional load.

`MD-Flexible` relies on the halo particle strategy due to force calculations only being done by particles owned by a given sub-domain/container, which is a drawback of using `AutoPas`. Therefore, `MD-Flexible's` domain decomposition strategy is largely guided by the capabilities and restrictions of `AutoPas`.

`MD-Flexible` employs adaptive domain decomposition, which dynamically adjusts the size and boundaries of a sub-domain. This ensures that the computational workload remains evenly distributed, even in the presence of inhomogeneous particle distributions, or evolving system states [5].

2.6.2. Regular Grid Decomposition

Regular Grid Decomposition partitions the simulation domain into a fixed grid of regular sub-domains (as depicted in Figure 2.5), with each sub-domain assigned to a single process. This method is straightforward to implement and offers the advantage that neighbour lists, which keep track of the neighbouring sub-domains for communication, only need to be calculated once, since in a regular grid they remain constant throughout the simulation. Furthermore, during load balancing, the grid structure does not require reconstruction as it is sufficient to shift the division planes between sub-domains to adjust workload distribution. However, this simplicity comes at the major cost of flexibility. Because the grid structure is fixed,

sub-domains cannot be resized independently, which can lead to suboptimal performance and unused computational potential.

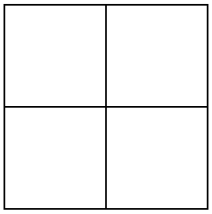


Figure 2.5.: A simple example of domain decomposition using a fixed regular grid. The 2D domain is decomposed into four sub-domains by dividing once along the x-axis and once along the y-axis. Adapted from [8].

In `MD-Flexible`, we currently use two methods for load balancing: ALL-library (see Subsection 2.6.3) and Inverse Pressure (see Subsection 2.6.4).

2.6.3. ALL-library

The ALL-library is a C++ library designed to provide load-balancing techniques for applications that utilise domain decomposition. It offers several methods for redistributing computational work dynamically during runtime and is purpose-built for simulation codes. It is developed and maintained by the Simulation Laboratory Molecular Systems of the Juelich Supercomputing Centre (JSC), Research Centre Juelich (Forschungszentrum Jülich GmbH) [9].

ALL provides various load balancing methods, such as:

- TENSOR
- STAGGERED
- HISTOGRAM
- FORCEBASED
- VORONOI

In `MD-Flexible`, the Tensor Method has been utilised to balance the load due to its simplicity and compatibility with the existing regular grid decomposition. This method works by trying to equalise the load across Cartesian directions independently, offering fast convergence without the overhead of full structural reconfiguration.

It is important to note that while the Tensor Method offers straightforward integration with the existing regular grid decomposition, other methods, such as Staggered, have demonstrated superior load balancing performance in other similar projects. However, supporting these methods would require a significant redesign of the current method of domain decomposition in `MD-Flexible`.

2.6.4. Inverse Pressure Load Balancer

The *Inverse Pressure Load Balancer* is another method for redistributing workload across sub-domains in `MD-Flexible`. It was first introduced in [5]. The approach is based on an

analogy to air pressure: two adjacent sub-domains are treated as volumes separated by a movable wall. If one sub-domain performs more work, and therefore has higher pressure, the wall is shifted towards the sub-domain with the lower workload.

For two neighbouring sub-domains A and B, the new boundary position is given by

$$x_1' = \frac{w_B x_2 + w_A x_0}{w_A + w_B},$$

where w_A and w_B denote the measured workloads of the two sub-domains. To avoid instabilities, boundaries are shifted only halfway towards this computed position. This procedure is illustrated in Figure 2.6.

In practice, the algorithm operates locally between adjacent planes along the x-, y-, and z-axes. While not globally optimal, this approach effectively reduces load imbalance with modest communication overhead.

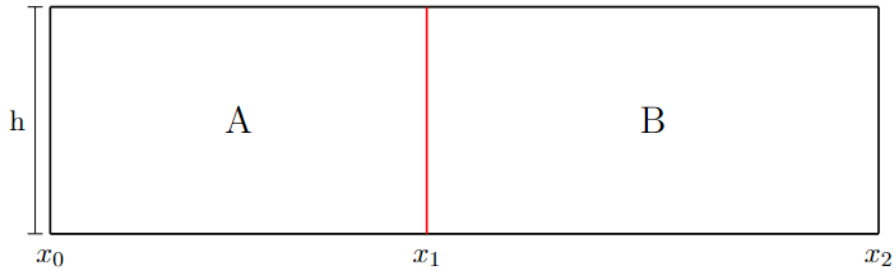


Figure 2.6.: Base scenario of the Inverse Pressure Load Balancing Method. Sub-domains A and B exert different pressures on a movable wall (red line). h denotes the height of the domains, while x_0, x_1, and x_2 represent the coordinates required to calculate their widths [5].

2.6.5. Workload Constraints

For both the ALL-library methods and the Inverse Pressure Load Balancer, the workload value ($w_a + w_b$) must always remain strictly positive. If $w_a + w_b = 0$ were permitted, divisions by zero would occur in the respective balancing formulas, leading to undefined behaviour.

`MD-Flexible` currently employs *computational time* as the measure of workload (w), but other parameters such as particle counts (see Chapter 6) or interaction counts could in principle be incorporated to improve load balancing.

Part II.

Related Work

3. Grid-Based Decompositions

This section discusses alternative domain decomposition strategies employed in other simulation software. As discussed in Subsection 2.6.2, `MD-Flexible` currently uses regular grid decomposition. Exploring other decomposition methods is important, as they may provide more efficient partitioning of the simulation domain, enabling better utilisation of computational resources and potentially reducing overall runtime.

3.1. Staggered Grids

A staggered grid decomposition refers to a spatial partitioning in which the boundaries between sub-domains are not fixed to a regular, uniform grid, as in the Regular Grid approach described in Subsection 2.6.2. Instead, the boundaries can be shifted independently in each dimension. This staggering enables each sub-domain's volume to be adjusted dynamically during a simulation, providing finer control over workload distribution.

Such flexibility is advantageous for dynamic load balancing, as sub-domains can expand or contract in response to changes in computational cost more freely. For instance, if particle density becomes uneven due to molecular motion, boundary adjustments can help maintain a balanced distribution of computational work. Unlike a regular grid, a staggered decomposition allows these adjustments without requiring the entire grid to change, which can be particularly beneficial in large-scale parallel simulations where small changes can significantly reduce load imbalance between ranks.

However, a staggered grid introduces additional complexity in determining neighbouring sub-domains. When domain boundaries are not perfectly aligned, neighbouring sub-domains may have overlapping or offset regions that require recalculating communication patterns at runtime. This contrasts with regular grids, where neighbouring relationships are fixed and computed only once. Consequently, the potential performance benefits of staggered grids must be weighed against the overhead introduced by these repeated calculations, which will affect MPI communication.

Such a staggered grid approach is implemented alongside regular grid in GROMACS [10], which enables effective dynamic load balancing. In practice, GROMACS has reported significant performance improvements from incorporating staggered decomposition.

3.2. Recursive Coordinate Bisection (RCB)

Recursive Coordinate Bisection (RCB) is a geometric partitioning algorithm where we recursively divide the simulation domain into sub-domains by introducing cutting planes orthogonal to one of the coordinate axes [11].

At each step, the axis along which the set of objects (e.g., particles) is most dense is selected as the splitting direction. The cut is placed such that the workload is balanced as

evenly as possible between the resulting sub-domains. This process is repeated recursively until a user-specified number of sub-domains is reached.

Although RCB was initially designed for partition counts that are powers of two, it can be generalised to any number of partitions by adjusting cut positions. Modern implementations, such as in the Zoltan library (see Section 3.3), extend RCB to support multiple balancing weights, allowing load balancing with respect to several criteria (e.g., force computation cost, communication cost) simultaneously.

RCB is one such algorithm that can result in a Staggered Grid Decomposition, as described in Section 3.1, since it adjusts sub-domain boundaries independently at each recursion level.

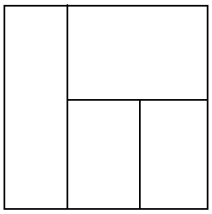


Figure 3.1.: Example of Recursive Coordinate Bisection (RCB) producing a staggered grid decomposition of the simulation domain. Adapted from [8].

3.3. Zoltan Load Balancer

The Zoltan library is an easy-to-use software toolkit developed at Sandia National Laboratories to address the challenges of load balancing and data management in large-scale, dynamic parallel simulations [12].

Zoltan is designed to provide a unified, data-structure-neutral interface for partitioning work across processors. It supports three primary classes of parallel partitioning algorithms:

- **Geometric methods** (e.g., Recursive Coordinate Bisection, see Section 3.2, and Hilbert Space-Filling Curves) that perform fast decompositions based on the spatial distribution of objects.
- **Graph-based methods** that represent objects as graph vertices and their adjacencies as edges, enabling load balancing while preserving mesh connectivity in both structured and unstructured domains.
- **Hypergraph-based methods** that generalise graph models to capture complex, irregular communication patterns with higher fidelity, improving load balance in highly non-uniform simulations.

An important feature of Zoltan is its object-based interface, which operates on application-defined objects such as particles or meshes, each identified by a unique global identifier. Applications provide Zoltan with object information via callback functions, including global and local IDs, coordinates, and optional weight values for multi-criteria balancing. This design allows integration with existing application data structures without imposing rigid format requirements.

By allowing the user to switch between partitioning strategies with minimal code changes, Zoltan enables performance tuning for different problem types and architectures. Its flexibility and broad algorithmic support make it widely applicable in high-performance computing contexts, including molecular dynamics simulations.

3.4. Empirical Successes

The Zoltan library as a load balancer has demonstrated superior performance compared to most native in-house load balancers as well as other widely used industry standards such as ParMETIS, as shown in [13]. ParMETIS is another commonly employed load balancer and has been utilised in our university's projects, including Load Balancing Earthquake Simulations [14].

Figure 3.2 illustrates that Zoltan performs on par with, or better than, ParMETIS in a comparable simulation project, Vlasiator. Furthermore, since Zoltan now includes ParMETIS as an optional interface, adopting Zoltan allows access to even more load balancing strategies through a single, unified interface.

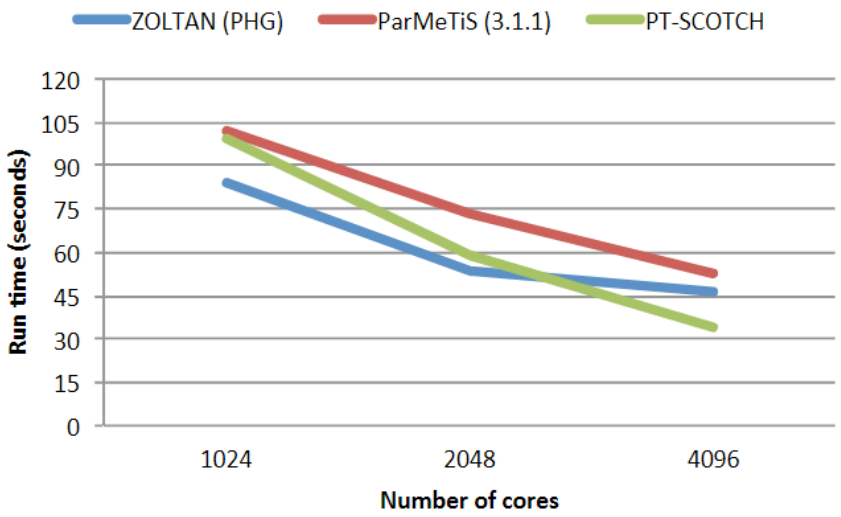


Figure 3.2.: Performance comparison of Zoltan and ParMETIS in Vlasiator [13].

3.5. Limitations and Challenges

One of the main challenges in implementing Zoltan, ParMETIS, or most other prominent load balancers within `MD-Flexible` lies in the current usage of Regular Grid Decomposition. Both Zoltan and ParMETIS, particularly for their most efficient partitioning strategies, often require or perform better with Staggered Grid Decomposition, as discussed in Section 3.1.

As noted in Subsection 2.6.2, this limitation stems from the `AutoPas` framework, which is fundamentally designed around regular grids. Integrating a staggered grid-based load balancing approach would therefore require substantial modifications to both `AutoPas` and

`MD-Flexible`. In particular, changes would be necessary in the management of sub-domain boundaries, neighbouring particles, and particle exchanges to ensure correct force calculations and consistent simulation results.

This limitation means that, while Zoltan and ParMETIS offer advanced load-balancing strategies, their direct adoption in `MD-Flexible` is not straightforward. Addressing this would involve a major architectural overhaul, carefully redesigning the interaction between neighbouring sub-domains and particle management. Consequently, while the potential performance benefits are significant, they come with substantial development and validation effort.

Part III.

Implementation and Evaluation

4. Experimental Setup

All experiments for MPI-based optimisation (Chapter 5) and parameter-tuned load balancing (Chapter 6) were conducted on the Linux cluster CoolMUC-4 at the Leibniz Supercomputing Centre. The compute nodes are equipped with Sapphire Rapids (Intel(R) Xeon(R) Platinum 8480+) processors. Simulations used the Exploding Liquid Scenario (see Appendix 8.1), with identical initial conditions to ensure comparability. Performance metrics, including total accumulated time, force update time, and communication overhead, were collected using built-in `MD-Flexible` timers, while MPI-related data was gathered using TAU (Section 2.5). TAU's own Paraprof tool was used for visualisation of Figures 5.4, 5.3, and 5.5.

Parameters such as `load-balancer` in Chapter 5 and `computation-load` in Chapter 6 were varied across test cases to evaluate their impact on performance.

Experiments were conducted on a single node with different task configurations:

- 2 tasks per node, 56 CPUs per task
- 4 tasks per node, 28 CPUs per task
- 8 tasks per node, 14 CPUs per task
- 16 tasks per node, 7 CPUs per task

This configuration was chosen so that the product of the number of tasks and CPUs per task equals 112, matching the maximum CPU limit per job on the Slurm partition `cm4_inter`. Consequently, as the number of tasks per node increases, the number of CPUs allocated per task decreases. Since each task relies on OpenMP threads for intra-task parallelisation, reducing CPUs per task limits the amount of parallelism available to each task. This explains why the total computational time observed in the graphs below tends to increase with a higher number of tasks per node.

The following commit IDs correspond to the specific changes relevant for reproducing the results:

- **Commit** `3f6a7ac19d9d` – Modified MPI communication pattern to use non-blocking receives and removed `iProbe` as discussed in Chapter 5.
- **Commit** `fffb8587e5f77` – Improved timing metrics and added `particleCount` as an additional computation load option as discussed in Chapter 6.

Including these commits ensures that the MPI optimisations and parameter-tuned load balancing modifications can be replicated exactly for correctness and performance evaluation.

5. MPI-Based Optimization

5.1. Motivation and Rationale

Profiling of the simulation revealed that a substantial portion of the total accumulated time, up to approximately 25% (see Table 8.1), was spent in MPI communication. The most time-consuming phases were:

1. exchanging emigrant particles between neighbouring domains, and
2. exchanging halo particles required for force calculations.

These stages, therefore, represent promising targets for performance optimisation.

A complete redesign of the particle migration and halo exchange mechanisms would require extensive modifications across both the `MD-Flexible` and `AutoPas` codebases, which was beyond the scope of this thesis. Instead, the optimisation efforts were concentrated on improving the efficiency of the existing MPI communication patterns in `MD-Flexible`. By refining the message-passing strategy, communication overhead can be reduced without altering the core algorithms for particle exchange, thereby enhancing scalability and overall runtime performance.

5.2. Implementation Details

5.2.1. Original Implementation

In the original implementation, halo particle exchange and particle migration both relied on the function `sendAndReceiveParticlesLeftAndRight`, which was invoked once per spatial dimension by each sub-domain. Thus, in a three-dimensional simulation, the function was called a total of six times in each iteration: three times for halo exchange and three times for particle migration.

For a given dimension and sub-domain, this function performs communication with up to two neighbouring sub-domains: the left and the right neighbours. The communication sequence is as follows:

1. Asynchronously send particles to the left neighbour using `MPI_Isend`.
2. Asynchronously send particles to the right neighbour using `MPI_Isend`.
3. Probe for incoming data from the left neighbour using `MPI_Probe`, then receive it via `MPI_Recv`.
4. Probe for incoming data from the right neighbour using `MPI_Probe`, then receive it via `MPI_Recv`.
5. Complete all outstanding send requests with `MPI_Waitall`.

This sequence corresponds to a total of seven MPI operations per call:

$$\texttt{Isend(left)} + \texttt{Isend(right)} + \texttt{Probe(left)} +$$
$$\texttt{Recv(left)} + \texttt{Probe(right)} + \texttt{Recv(right)} + \texttt{Waitall}.$$

Since the function is invoked six times per iteration in three dimensions, the original implementation executes a total of

$$6 \times 7 = 42 \text{ MPI operations per iteration per sub-domain.}$$

5.2.2. New Implementation

The new implementation restructures `sendAndReceiveParticlesLeftAndRight` to consolidate all left- and right-neighbour exchanges into a single routine within `ParticleCommunicator.cpp`.

A new parameter, `dimension`, was added to uniquely tag messages by dimension index (0, 1, or 2). These tags are derived as `dimension * 2 + offset`, where `offset` is `0` for left and `1` for right. This ensures correct message matching without relying on `MPI_Probe`, which reduces latency and eliminates the need for blocking size queries.

To further reduce communication overhead, several other optimisations were introduced:

- **Reusable Buffers:** Dedicated send and receive buffers for left and right neighbours are maintained across calls, avoiding repeated memory allocations and deallocations. This not only reduces heap overhead but also improves cache reuse.
- **Pre-serialisation with Particle Counts:** Each message begins with an integer indicating the number of particles being sent, followed by their serialised data. This enables direct deserialisation of received messages without probing for message sizes.
- **Non-blocking point-to-point communication:** `MPI_Irecv` is posted to both neighbours *before* any sends, so receives are ready to match as soon as messages arrive (no probes needed). Afterwards, `MPI_Isend` is issued to both neighbours. Completion is handled with two `MPI_Waitall` calls.
- **Reduced MPI Calls:** The original implementation required up to seven MPI calls per dimension (`Isend`, `Isend`, `Probe`, `Recv`, `Probe`, `Recv`, `Waitall`). The new approach reduces this to six (`Irecv`, `Irecv`, `Isend`, `Isend`) and two `Waitall` calls, eliminating all probes and blocking receives.

Deserialisation now operates directly on the received buffers, using the stored particle count to correctly extract payload segments. By combining tag-based matching, persistent buffers, and fully non-blocking communication, the new implementation should reduce message latency and lower the per-dimension communication cost in large-scale simulations.

Since the function is still invoked six times per iteration in a three-dimensional simulation, the new implementation now performs a total of

$$6 \times 6 = 36 \text{ MPI operations per iteration per sub-domain,}$$

representing a reduction of approximately 14% in the number of communication calls compared to the original 42 operations.

By ensuring all MPI calls are non-blocking, the new implementation eliminates idle waiting time. This design also opens the possibility of overlapping communication with computation, e.g., halo exchange could be performed concurrently with force updates. Although not implemented here due to required structural changes in `MD-Flexible` and `AutoPas`, this remains a promising direction for future work.

5.3. Experimental Results

Figures 5.1 and 5.2 compare the runtime performance of the original MPI implementation against the new version described above. Contrary to expectations, the new implementation consistently resulted in higher runtimes than the baseline. This indicates that, despite reducing the number of MPI calls from 42 to 36 per iteration per sub-domain, the communication efficiency did not improve.

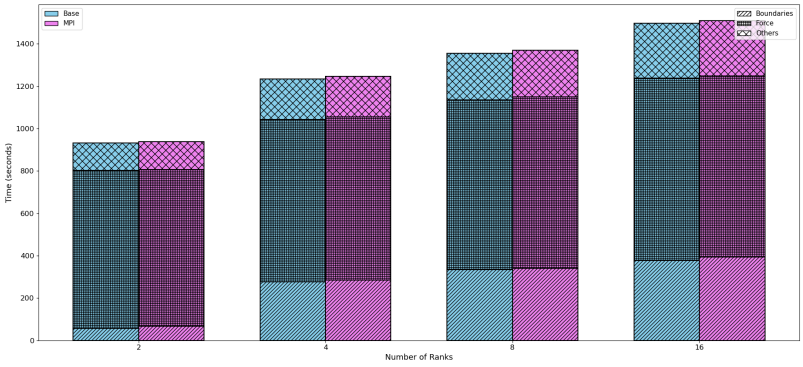


Figure 5.1.: Performance Breakdown of the baseline and new MPI implementations by Ranks when using the ALL Load Balancer (see Subsection 2.6.3). The new implementation underperforms relative to the baseline in all cases.

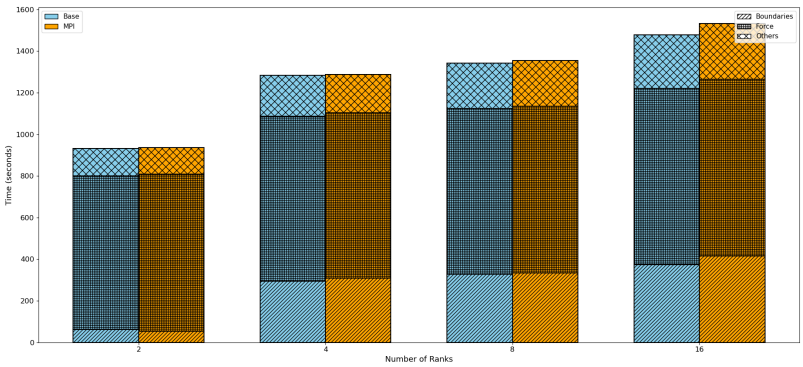


Figure 5.2.: Performance Breakdown of the baseline and new MPI implementations by Ranks when using the Inverse Pressure Load Balancer (see Subsection 2.6.4). The new implementation underperforms relative to the baseline in all cases.

5.4. Qualitative Evaluation

To better understand the performance discrepancy, detailed MPI profiling was carried out using TAU (Section 2.5). Figures 5.3, 5.4, and 5.5 illustrate the distribution of communication costs across MPI operations.

The baseline implementation (Figure 5.3) revealed that `MPI_Probe` dominated communication costs, acting as the main bottleneck. In the optimised version, the probes were successfully eliminated, but the overhead was effectively shifted to `MPI_Waitall` (Figure 5.4). As a result, the overall communication cost remained high, negating the expected benefits of reducing the number of MPI calls and non-blocking communication pattern.

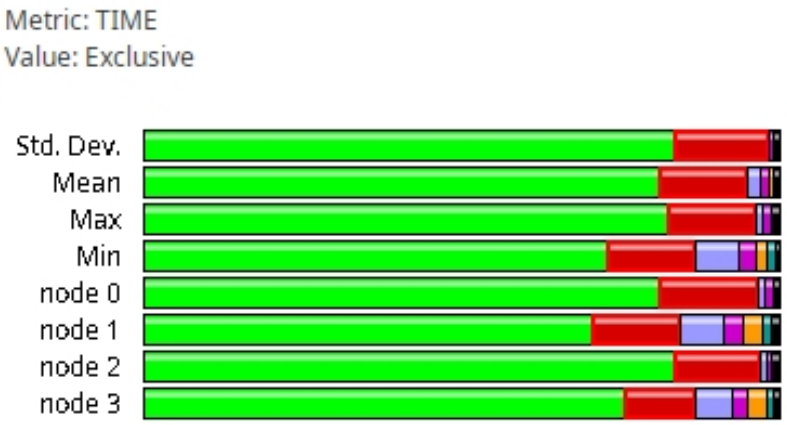


Figure 5.3.: Normalised TAU profile of the baseline MPI implementation when tested with ALL load balancer (legend in Figure 5.5). The `MPI_Probe` operation dominates communication costs.

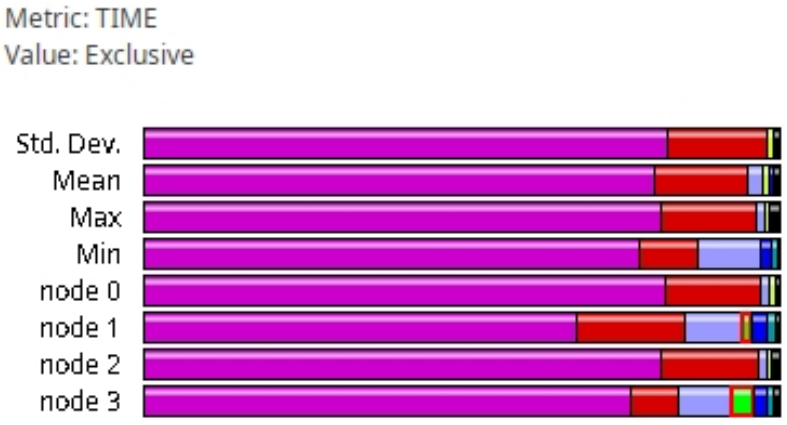


Figure 5.4.: Normalised TAU profile of the optimised MPI implementation when tested with ALL load balancer (legend in Figure 5.5). Although probes are eliminated, their cost is replaced by `MPI_Waitall`, resulting in no reduction in total communication time.

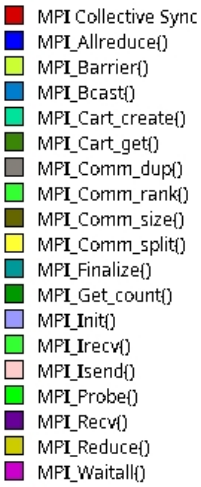


Figure 5.5.: Legend for the TAU profiling results, mapping MPI functions to their corresponding visual representation in Figures 5.3 and 5.4.

5.5. Interpretation of Results

The profiling results demonstrate that the reduction in the number of MPI calls alone does not guarantee performance improvement. While probes were removed, the new design introduced substantial synchronisation overhead due to `MPI_Waitall`. Thus, the gains from fewer MPI operations were offset by the cost of global synchronisation, leading to slower runtimes overall.

However, this experiment should not be regarded as a failure. Firstly, as discussed in Subsection 5.2.2, the revised communication pattern is now fully non-blocking, thereby enabling the possibility of overlapping communication with computation, a capability absent from the previous probe-based design. Secondly, the results indicate that the original MPI implementation was already relatively well-optimised for the existing halo and particle migration schemes. Consequently, substantial performance improvements in boundary handling will likely require structural changes to the design of halo exchanges and particle migrations themselves, rather than refinements to the MPI communication layer alone.

Utilising the potential of the new non-blocking communication scheme by overlapping communication and computation would, however, require a significant restructuring of both `AutoPas` and `MD-Flexible`. Such a redesign would necessitate changes in how computation phases, such as force calculation, are organised relative to communication, but in theory, it could deliver considerably better performance compared to the current sequential design.

6. Parameter-Tuned Load Balancing

6.1. Motivation and Rationale

As demonstrated in Chapter 5, attempts to improve performance through MPI-level modifications gave sub-optimal results. This prompted a more thorough evaluation of the existing load balancing strategies in the system. During this evaluation, a key area for improvement was identified, namely, how computational load was measured and subsequently used to guide load-balancing decisions.

In the original implementation, computational time was treated in a simplistic and fragmented manner, potentially leading to suboptimal domain adjustments. This raised concerns about the accuracy and reliability of the load balancing and, hence, the domain resizing. A more comprehensive strategy was needed to better reflect actual computational demand.

To address this, the idea to refine and expand the parameters considered during load balancing was put forward. Rather than relying solely on the computation load, the proposal focused on using various targeted time intervals within the simulation for load balancing. It was assumed that using these various values for load balancing would yield results that would be beneficial, for instance, in scenarios where force computations represent the primary performance bottleneck, the load balancer could prioritise force-related timings, or when communication overhead was the dominant factor, MPI-specific timings could serve as the guiding metric for domain adjustments.

6.2. Implementation Details

6.2.1. Original Implementation

In the previous version of the simulation loop (see Algorithm 1), load-balancing was driven exclusively by the `computationalLoad` timer. Although straightforward, this approach failed to capture the full complexity of the simulation's performance characteristics.

Preliminary profiling (see Section 8.1) showed that a significant fraction of runtime was spent not only in force calculations, but also in particle migration, boundary reflections, and halo exchanges. These components can vary substantially depending on the simulation scenario and number of sub-domains, yet their contributions were invisible to the existing load-balancing strategy. As a result, the original implementation often overlooked important performance bottlenecks, limiting its ability to achieve well-balanced domain decompositions.

6.2.2. New Implementation

The updated version of the simulation loop (see Algorithm 2) introduces a configurable, parameter-tuned load metric for balancing (as described in Algorithm 3). This system allows

users to specify a load metric in the simulation configuration file (as described in Listing 8.1), enabling targeted optimisation based on the specific workload characteristics of each simulation run.

One key enhancement is the introduction of a **particle count** load balancer. While the previous implementation only used time-based metrics, the new design leverages the existing flexibility of the load balancing framework to accept any scalar metric (see Subsection 2.6.5). This opens the door to using particle counts directly as a proxy for computational demand, which is particularly useful in particle-based simulations where load is strongly correlated with particle density.

It should be noted, however, that this new metric is limited to the raw particle count and does not incorporate further information about particle properties such as mass, velocity, or acting forces. For this reason, it may be described as a *pseudo particle-aware* load balancer. In contrast, some simulation projects have explored *true particle-aware* load balancing approaches, where particle properties are explicitly considered to predict boundary shifts more accurately.

In the new implementation, the following load metrics are supported:

- **CompleteCycle** – A comprehensive metric that aggregates all major timers, including `computationalLoad`, `loadBalancing`, `haloParticleExchange`, `reflectParticles`, and `migratingParticleExchange`. Suitable for capturing overall simulation effort.
- **ForceUpdate** – Focuses exclusively on the `forceUpdateTotal` timer. Ideal for simulations where force computation is the primary performance bottleneck.
- **MPICommunication** – Sums the `haloParticleExchange` and `migratingParticle Exchange` timers. Useful when communication overhead dominates runtime.
- **ParticleCount** – Uses the number of owned particles per sub-domain as the load metric. Provides an indirect yet effective heuristic for estimating computational and communication cost.
- **Default (Computational Load)** – Falls back to the original `computationalLoad` timer if no specific metric is selected.

This extensible design lays the foundation for further research into adaptive or dynamic load balancing strategies, potentially enabling the system to switch metrics mid-simulation based on real-time profiling.

It is important to note that, due to the workload restrictions described in Subsection 2.6.5, an explicit check is performed when updating the computational load. This is necessary because, in the first iteration, some of the load-balancing metrics may evaluate to zero and only assume their meaningful values in later iterations.

6.3. Experimental Results

Figure 6.1 summarises the total computation time for the exploding liquid scenario. Experiments were run as described in Chapter 4. Results were obtained using only the `ALL` load balancer, since, as shown in Chapter 5, the `inverse pressure` load balancer consistently performs worse. The `Default` strategy serves as the baseline reference, as it is identical to the implementation prior to any changes.

Results show that **Particle Count** and **Force Calculation** performed the best, while **MPI Communication** was consistently the worst.

On average, the **Particle Count** metric was approximately `20%` faster than `Default`, while **MPI Communication** was `30%` slower. Interestingly, **Complete Cycle** was, on average, neither significantly better nor worse than `Default`.

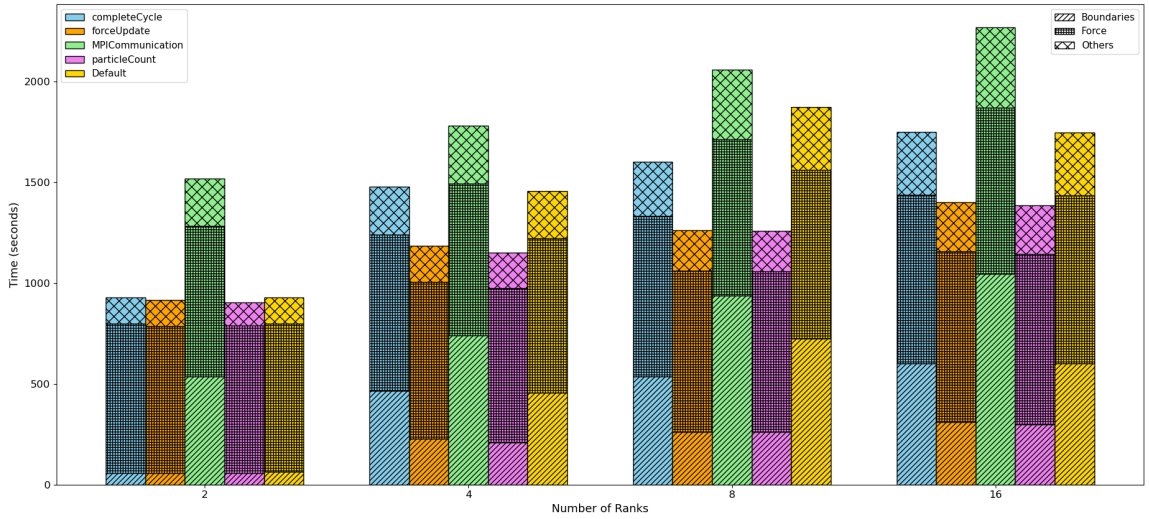


Figure 6.1.: Performance Breakdown of different Load Balancing Strategies by Ranks.

6.4. Qualitative Evaluation

To better understand these results, we can examine how the boundaries are drawn using the ParaView visualisation tool. The baseline `Default` option decomposes the domain such that, as shown in Figure 6.2, boundaries neither follow the gaps nor attempt to balance the densest parts of the simulation, acting as a perfect middle ground. By contrast, the `Particle Count` option decomposes the domain so that, as seen in Figure 6.3, each sub-domain contains almost the same number of owned particles. The `MPI Communication` option, illustrated in Figure 6.4, biases decomposition strongly towards empty gaps in the domain and attempts to minimise particle exchange between neighbours, which is its explicit optimisation goal.

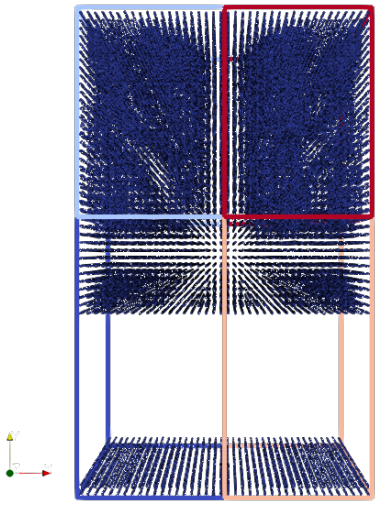


Figure 6.2.: Domain Decomposition of Exploding Liquid after 500 time-steps using Default Metric and All Load Balancer.

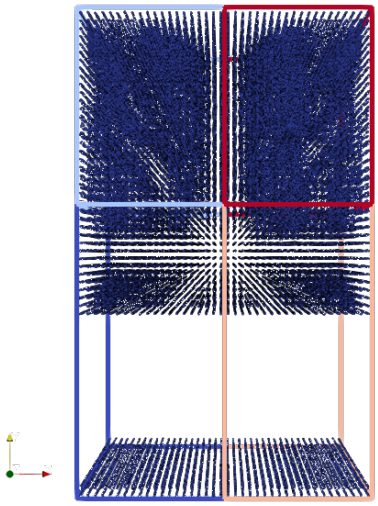


Figure 6.3.: Domain Decomposition of Exploding Liquid after 500 time-steps using Particle-Count Metric and All Load Balancer.

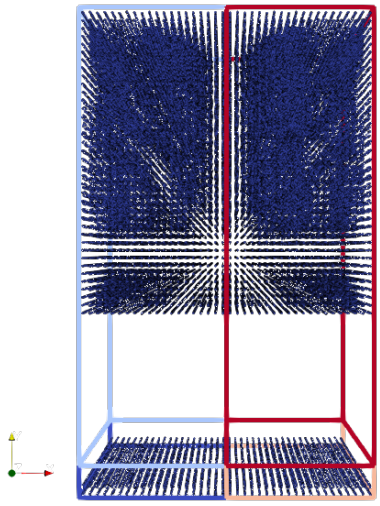


Figure 6.4.: Domain Decomposition of Exploding Liquid after 500 time-steps using MPI-Communication Metric and All Load Balancer.

6.5. Interpretation of Results

Among the evaluated strategies, `Particle Count` emerged as the most effective. This is unsurprising, as particle distribution directly reflects the main source of computational load in MD simulations.

The `Force Calculation` metric also performed well, indicating that timing-based measurements can be a viable alternative when particle counts alone do not fully capture imbalance.

By contrast, `MPI Communication` was the weakest performer. Its poor results stem from the fact that it optimises only for minimising message passing, without addressing the actual root cause of imbalance: the uneven distribution of particles. While such a strategy could still prove useful in communication-dominated scenarios or on hardware highly optimised for message passing, in this setting, it consistently underperformed.

The `Complete Cycle` strategy also failed to deliver improvements. By combining all timing metrics with equal weight, it effectively overemphasised communication costs, which were not the limiting factor in these experiments. This suggests that weighted or dynamically tuned combinations of metrics could provide a more balanced approach.

Finally, the `Default` strategy remained a robust baseline, especially for users who prefer not to make assumptions about the source of imbalance.

Overall, these results highlight that strategies grounded in particle distribution, particularly `Particle Count`, are the most reliable. A promising direction for future work is the development of a truly *particle-aware* load balancer that accounts not only for particle numbers but also for properties such as mass, velocity, or force. Such an approach could anticipate shifts in distribution and pre-emptively adapt domain boundaries. Looking further ahead, `MD-Flexible` could even be extended with an auto-tuning mechanism that

dynamically selects the most relevant metric at runtime, paving the way for a self-optimising load balancer.

Part IV.

Conclusion

7. Key Findings and Future Directions

7.1. Summary and main conclusions

This thesis evaluated two targeted directions to improve parallel performance of `MD-Flexible`: (1) an MPI communication redesign for halo exchange and particle migration, and (2) a parameter-tuned load-balancer with improved metrics. Experiments produced two clear conclusions:

- **MPI restructuring alone did not reduce runtime.** Replacing probe-based communication with non-blocking receives, tagged messages and persistent buffers reduced MPI call count (42→36) but shifted cost from `MPI_Probe` to `MPI_Waitall`. Total runtimes increased, showing that fewer MPI calls are not sufficient if communication remains serialised with computation.
- **Metric-aware load balancing is effective.** Allowing the balancer to use different scalar metrics produced immediate gains: the simple `ParticleCount` metric and `ForceUpdate` timing outperformed the previous default metric in the tested scenario (ParticleCount $\approx 20\%$ faster while the MPI-focused metric performed worse by $\approx 30\%$). This demonstrates that choosing a metric closely aligned to particles yield better runtimes.

Taken together, these results show that (a) improving the MPI layer is a necessary step, but (b) the highest practical gains are obtained when load metrics reflect the true computational bottleneck. The MPI changes are therefore valuable primarily because they enable overlap opportunities that were impossible with the old blocking design.

7.2. Future work

1. **Evaluate staggered decompositions and external balancers.** Study the engineering cost and payoff of supporting staggered grids (and integrating Zoltan/ParMETIS) in `AutoPas`/`MD-Flexible` for possibly better load balancing.
2. **Overlap communication with computation:** Reorganise time step phases so non-blocking halo particle exchanges progress concurrently with force updates. This is the most direct way to convert `MPI_Waitall` overhead into hidden latency.
3. **Adaptive metric selection / lightweight autotuning.** Implement a runtime sampler that chooses or weights metrics (ParticleCount, ForceUpdate, MPICommunication) dynamically based on short profiling windows.
4. **Develop a particle-aware load balancer.** Extend the load balancing strategy to consider particle properties such as force, velocity, or mass, rather than relying only

on particle counts. This would allow boundaries to be predicted more accurately and adapted to future particle movements, improving overall balance.

Part V.

Appendix

8. Additional Material

8.1. Baseline MD-Flexible Results

The following results show MD-Flexible running with 16 tasks and 16 CPUs.

Measurement	**Time (ns / s / %)**
Total accumulated	1495949011504 ns (1495.949s)
Initialization	4570724568 ns (4.571s) = 0.306%
Simulate	1491053813797 ns (1491.054s) = 99.673%
PositionUpdate	19620779091 ns (19.621s) = 1.316%
UpdateContainer	128564523870 ns (128.565s) = 8.622%
Boundaries	377323632697 ns (377.324s) = 25.306%
HaloParticleExchange	130926761622 ns (130.927s) = 34.698%
ReflectParticlesAtBoundaries	3644555 ns (0.004s) = 0.001%
MigratingParticleExchange	246396871075 ns (246.397s) = 65.301%
ForceUpdateTotal	859923696552 ns (859.924s) = 57.672%
ForceUpdateNonTuning	859894617097 ns (859.895s) = 99.997%
VelocityUpdate	19199932222 ns (19.200s) = 1.288%
Vtk	659446019 ns (0.659s) = 0.044%
LoadBalancing	85677735685 ns (85.678s) = 5.746%
One iteration	1491053813 ns (1.491s) = 0.100%
Total wall-clock time	93476314761 ns (93.476s) = 6.249%
Tuning iterations	0 / 1000 = 0%
MFUPs/sec	0.00348868
Mean Rebuild Frequency	3.34448

Table 8.1.: Performance results of MD-Flexible with 16 tasks and 16 CPUs.

8.2. Code and YAML Listings

This section includes selected code and configuration listings used throughout the thesis.

8.2.1. YAML Test Configuration

The YAML used for testing. Parameters such as ‘load-balancer‘ and ‘computation-load‘ were varied based on the test cases.

Listing 8.1: Test Configuration YAML

```
traversal                                :  [lc_c08]
```

```
newton3                                 :  [enabled]
data-layout                             :  [SoA]

# Simulation General Parameters
iterations              : 100
deltaT                  : 0.00182367
boundary-type           : [periodic, periodic, periodic]
log-level               : info
tuning-interval         : 1000

# Particle Interaction Parameters
verlet-rebuild-frequency : 10
verlet-skin-radius      : 0.2
functor                 : Lennard-Jones
cutoff                  : 2
cell-size               : [1]

# Domain and Load Balancing
box-min                 : [0, 0, 0]
box-max                 : [15, 60, 15]
load-balancer           : all
#computation-load        : completeCycle
load-balancing-interval : 5
subdivide-dimension     : [true, true, true]

# Particle Properties (Sites)
Sites:
  0:
    epsilon             : 1.0
    sigma               : 1.0
    mass                : 1.0

# Particle Objects (Initial Configuration)
Objects:
  CubeGrid:
    0:
      particle-type-id  : 0
      box-length        : [14, 6, 14]
      particles-per-dimension : [40, 40, 40]
      bottomLeftCorner  : [0.5, 27, 0.5]
      particle-spacing  : 1.0
      velocity          : [0, 0, 0]

# Output Control
vtk-filename            : vtk
vtk-write-frequency     : 5
```

8.2.2. Simulation Algorithms

The following pseudocode outlines key simulation loops and computational load estimation logic used in the implementation.

Algorithm 1: Simplified simulation loop with emphasis on timers

Data: Simulation configuration, domain decomposition, container
Result: Advances simulation through iterations while tracking load and timing

```
1  startTimer(simulate)
2  while needsMoreIterations() do
3      startTimer(computationalLoad)
4      if configuration Δt ≠ 0 and not paused then
5          update positions and reset forces
6          if Multisite mode enabled then
7              update quaternions
8      startTimer(updateContainer)
9      emigrants ← update container
10     stopTimer(updateContainer)
11     if time step Δt ≠ 0 and not paused then
12         computeLoad ← stopTimer(computationalLoad)
13         if iteration mod loadBalancingInterval == 0 then
14             startTimer(loadBalancing)
15             perform load balancing and adjust domain based on computeLoad
16             stopTimer(loadBalancing)
17         startTimer(migratingParticleExchange)
18         exchange emigrants across domains
19         stopTimer(migratingParticleExchange)
20         startTimer(reflectParticles)
21         reflect particles at boundaries
22         stopTimer(reflectParticles)
23         startTimer(haloExchange)
24         exchange halo particles
25         stopTimer(haloExchange)
26         startTimer(computationalLoad)
27     compute and update interaction forces
28     if pauseSimulationDuringTuning is enabled then
29         update simulation pause state
30     if configuration Δt ≠ 0 and not paused then
31         update velocities
32         if Multisite mode enabled then
33             update angular velocities
34         update thermostat
35     stopTimer(computationalLoad)
36     if not paused then
37         increment iteration counter
38 stopTimer(simulate)
```

Algorithm 2: Simulation loop with timer-based computational load estimation

Data: Simulation configuration, domain decomposition, container
Result: Advances simulation through iterations while tracking load and timing

```
1  startTimer(simulate)
2  while needsMoreIterations() do
3      startTimer(computationalLoad)
4      if configuration Δt ≠ 0 and simulation not paused then
5          updatePositionsAndResetForces()
6          if Multisite mode enabled then
7              updateQuaternions()
8          startTimer(updateContainer)
9          emigrants ← updateContainer()
10         stopTimer(updateContainer)
11         stopTimer(computationalLoad)
12         totalTime ← calculateComputationLoad()
13         if load mode == particle count then
14             computateLoad ← totalTime
15         else
16             computateLoad ← totalTime − previousTimerValue
17             previousTimerValue →← totalTime
18         if iteration % loadBalancingInterval == 0 then
19             startTimer(loadBalancing)
20             _domainDecomposition→update(computeLoad == 0 ?
                _timers.computationalLoad.getTotalTime() : computeLoad)
21             stopTimer(loadBalancing)
22         startTimer(migratingParticleExchange)
23         exchangeMigratingParticles(emigrants)
24         stopTimer(migratingParticleExchange)
25         startTimer(reflectParticles)
26         reflectParticlesAtBoundaries()
27         stopTimer(reflectParticles)
28         startTimer(haloParticleExchange)
29         exchangeHaloParticles()
30         stopTimer(haloParticleExchange)
31         startTimer(computationalLoad)
32     compute and update interaction forces
33     if pauseSimulationDuringTuning is enabled then
34         updateSimulationPauseState()
35     if configuration Δt ≠ 0 and simulation not paused then
36         updateVelocities()
37         if Multisite mode enabled then
38             updateAngularVelocities()
39         updateThermostat()
40     stopTimer(computationalLoad)
41     if not paused then
42         increment iteration counter
43 stopTimer(simulate)
```

Algorithm 3: Computation load timers based on configuration

Data: Simulation configuration, timer values, and particle container
Result: Returns scalar value representing computational load

```
1  switch computationLoad mode do
2      case completeCycle do
3          return sum of computationalLoad, loadBalancing,
             haloParticleExchange and migratingParticleExchange timers
4      case forceUpdate do
5          return total time from forceUpdateTotal timer
6      case MPICommunication do
7          return sum of haloParticleExchange and migratingParticleExchange
             timers
8      case particleCount do
9          return number of owned particles in container
10     return total time from computationalLoad timer
```

List of Figures

2.1. Falling Drop simulation snapshot [5]. 5
2.2. Spinodal simulation snapshot [5]. 5
2.3. Exploding Liquid simulation snapshot [5]. 6
2.4. Illustration of the cut-off radius and skin width. The reference particle is shown in black; directly accessible particles are in blue; halo particles, which must be imported, are in red; and inaccessible particles are in grey. Adapted from [5]. 8
2.5. A simple example of domain decomposition using a fixed regular grid. The 2D domain is decomposed into four sub-domains by dividing once along the x-axis and once along the y-axis. Adapted from [8]. 9
2.6. Base scenario of the Inverse Pressure Load Balancing Method. Sub-domains A and B exert different pressures on a movable wall (red line). h denotes the height of the domains, while x_0, x_1, and x_2 represent the coordinates required to calculate their widths [5]. 10

3.1. Example of Recursive Coordinate Bisection (RCB) producing a staggered grid decomposition of the simulation domain. Adapted from [8]. 13
3.2. Performance comparison of Zoltan and ParMETIS in Vlasiator [13]. 14

5.1. Performance Breakdown of the baseline and new MPI implementations by Ranks when using the ALL Load Balancer (see Subsection 2.6.3). The new implementation underperforms relative to the baseline in all cases. 20
5.2. Performance Breakdown of the baseline and new MPI implementations by Ranks when using the Inverse Pressure Load Balancer (see Subsection 2.6.4). The new implementation underperforms relative to the baseline in all cases. 20
5.3. Normalised TAU profile of the baseline MPI implementation when tested with ALL load balancer (legend in Figure 5.5). The `MPI_Probe` operation dominates communication costs. 21
5.4. Normalised TAU profile of the optimised MPI implementation when tested with ALL load balancer (legend in Figure 5.5). Although probes are eliminated, their cost is replaced by `MPI_Waitall`, resulting in no reduction in total communication time. 21
5.5. Legend for the TAU profiling results, mapping MPI functions to their corresponding visual representation in Figures 5.3 and 5.4. 22

6.1. Performance Breakdown of different Load Balancing Strategies by Ranks. . 25
6.2. Domain Decomposition of Exploding Liquid after 500 time-steps using Default Metric and All Load Balancer. 26

6.3. Domain Decomposition of Exploding Liquid after 500 time-steps using ParticleCount Metric and All Load Balancer. 26
6.4. Domain Decomposition of Exploding Liquid after 500 time-steps using MPI-Communication Metric and All Load Balancer. 27

Bibliography

[1] Harold A. Scheraga, Mey Khalili, and Adam Liwo. Protein-folding dynamics: Overview of molecular simulation techniques. *Annual Review of Physical Chemistry*, 58(Volume 58, 2007):57–83, 2007.

[2] Zefan Yan, Rongzheng Liu, Bing Liu, Youlin Shao, and Malin Liu. Molecular dynamics simulation studies of properties, preparation, and performance of silicon carbide materials: A review. *Energies*, 16(3), 2023.

[3] Reza Bozorgpour. Computational explorations in biomedicine: Unraveling molecular dynamics for cancer, drug delivery, and biomolecular insights using lammps simulations, 2024.

[4] Fabio Alexander Gratl, Steffen Seckler, Nikola Tchipev, Hans-Joachim Bungartz, and Philipp Neumann. Autopas: Auto-tuning for particle simulations. In *2019 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, May 2019.

[5] Jacky Körner. Enabling massive parallelism for the autopas demonstrator md-flexible using adaptive domain decomposition. Master's thesis, Technical University of Munich, Oct 2021.

[6] Message Passing Interface Forum. *MPI: A Message-Passing Interface Standard Version 5.0*, June 2025.

[7] TAU: Tuning and Analysis Utilities. `http://www.cs.uoregon.edu/research/paracomp/tau/`.

[8] David A. Kopriva. A conservative staggered-grid chebyshev multidomain method for compressible flows. ii. a semi-structured method. *Journal of Computational Physics*, 128(2):475–488, 1996.

[9] R. Halver, S. Schulz, and G. Sutmann. ALL - A loadbalancing library, C++ / Fortran library. `https://gitlab.version.fz-juelich.de/SLMS/loadbalancing/-/releases`.

[10] Mark Abraham, Andrey Alekseenko, Brian Andrews, Vladimir Basov, Paul Bauer, Hugh Bird, Eliane Briand, Ania Brown, Mahesh Doijade, Giacomo Fiorin, Stefan Fleischmann, Sergey Gorelov, Gilles Gouaillardet, Alan Gray, M. Eric Irrgang, Farzaneh Jalalypour, Petter Johansson, Carsten Kutzner, Grzegorz Łazarski, Justin A. Lemkul, Magnus Lundborg, Pascal Merz, Vedran Miletić, Dmitry Morozov, Lukas Müllender, Julien Nabet, Szilárd Páll, Andrea Pasquadibisceglie, Michele Pellegrino, Nicola Piasentin, Daniele Rapetti, Muhammad Umair Sadiq, Hubert Santuz, Roland Schulz, Michael

Shirts, Tatiana Shugaeva, Alexey Shvetsov, Philip Turner, Alessandra Villa, Sebastian Wingbermühle, Berk Hess, and Erik Lindahl. Gromacs 2025.2 manual, May 2025.

[11] Berger and Bokhari. A partitioning strategy for nonuniform problems on multiprocessors. *IEEE Transactions on Computers*, C-36(5):570–580, 1987.

[12] Erik Boman, Karen Devine, Robert Heaphy, Bruce Hendrickson, Vitus Leung, Lee Ann Riesen, Courtenay Vaughan, Umit Catalyurek, Doruk Bozdag, William Mitchell, and James Teresco. *Zoltan 3.0: Parallel Partitioning, Load Balancing, and Data-Management Services; User's Guide*. Sandia National Laboratories, Albuquerque, NM, 2007. Tech. Report SAND2007-4748W `http://www.cs.sandia.gov/Zoltan/ug_html/ug.html`.

[13] Ata Turk, Cevdet Aykanat, G. Vehbi Demirci, Sebastian von Alfthan, and Ilja Honkonen. Improving the load balancing performance of vlasiator, 2011.

[14] David Jonathan Schneller. Evaluation of graph partitioning algorithms for load balancing of earthquake simulations. Master's thesis, Technical University of Munich, Mar 2019.